

Executive Code Documentation

Project: Rovedg Physics Study Platform

Branch prepared for main: `main`

Date: 2026-05-03

1. Executive Summary

This document explains the current codebase after the Flask redesign, local tooling work, project context system, smoke tests, and local artifact cleanup. The project keeps the original technology choices: Python, Flask, Jinja HTML templates, static CSS files, static PDFs, and JSON data files. No database, React migration, or external frontend framework was introduced.

The main goal was to make the platform easier to maintain and easier to study from. The site now behaves more like a study dashboard: students can start from notes, move into exercises, review exams, and consult PAU information from a clearer navigation model.

The project remains Render-compatible because the application entry point is still `app.py`, the Flask app object is still named `app`, and dependencies remain declared in `requirements.txt`.

Current repository state:

- The redesign and structural refactor are merged into `main`.
- Local Windows helper scripts are present for setup and launch.
- Pytest smoke tests are present under `tests/`.
- Project context files are present under `context/` to guide future AI-assisted work.
- `.gitignore` excludes generated Python caches, pytest caches, local virtual environments, Codex temporary files, local environment files, and common OS metadata.
- The active data model is still JSON files; there is still no database.

2. High-Level Architecture

The application has five practical layers:

- Flask application layer: `app.py` defines routes, helper functions, context values, JSON loaders, and session-protected pages.
- Data layer: `data/*.json` stores topics, quizzes, exams, and summaries without using a database.
- Template layer: `templates/` contains Jinja templates and reusable macros.
- Static layer: `static/css/` contains split CSS files, and `static/pdfs/` contains downloadable study materials.
- Tooling and documentation layer: `tests/`, local run scripts, `README.md`, `AGENTS.md`, and `context/` keep the project understandable and safer to maintain.

Request flow:

1. A browser requests a Flask route such as `/topics`.

2. Flask calls the matching route function in `app.py`.
3. The route loads JSON data when needed.
4. The route renders a Jinja template.
5. Jinja templates call helper functions such as `asset_url()` and reusable macros.
6. The browser receives HTML, CSS, Bootstrap assets, Font Awesome icons, and links to PDFs.

3. Modified Application File: `app.py`

`app.py` is still the project entry point. The redesign made it more expandable by introducing reusable data helpers, shared navigation metadata, and template context helpers.

Global object: `app`

Purpose:

- Creates the Flask application instance.
- Keeps compatibility with Render and Gunicorn via `gunicorn app:app`.

Inputs:

- No direct input. Flask initializes from the current module.

Outputs:

- A configured Flask application object.

Important behavior:

- The secret key now reads from the `SECRET_KEY` environment variable first.
- If `SECRET_KEY` is not set, it falls back to a local development key.

Constant: `BASE_DIR`

Purpose:

- Stores the absolute path of the project root.
- Avoids relying on whichever folder the process happens to run from.

Inputs:

- Uses `__file__`.

Outputs:

- Absolute filesystem path as a string.

Constant: `DATA_DIR`

Purpose:

- Points to the `data/` folder.
- Used by JSON-loading helpers.

Inputs:

- Uses `BASE_DIR`.

Outputs:

- Absolute filesystem path to `data/`.

Constant: `USERS`

Purpose:

- Keeps the existing simple username/password access model.

Inputs:

- None at runtime.

Outputs:

- Used by the `/login` route to validate credentials.

Notes:

- This is still intentionally simple.
- A future version should move credentials to environment variables or a safer authentication layer.

Constant: `NAV_ITEMS`

Purpose:

- Centralizes the main navigation structure.
- Prevents duplicating menu labels and icons across templates.

Inputs:

- Static list of dictionaries.

Outputs:

- Injected into templates through `inject_layout_context()`.

Each item contains:

- `endpoint`: Flask endpoint name, for example `topics`.
- `label`: Visible navigation text, for example `Apuntes`.
- `description`: Short explanation used in dashboard cards.
- `icon`: Font Awesome icon class.

4. New and Updated Functions in app.py

`login_required(f)`

Purpose:

- Protects pages that should only be visible after login.

Inputs:

- `f`: the route function being protected.

Outputs:

- A wrapped route function.

Behavior:

- If `session[ 'username' ]` is missing, the user is redirected to `/login`.
- If the session exists, the original route function runs normally.

Used by:

- `/topics`
- `/homework`
- `/quiz/<quiz_id>`
- `/submit-quiz/<quiz_id>`
- `/quiz-results`
- `/miscellaneous`

`data_path(filename)`

Purpose:

- Builds a safe absolute path to a file inside `data/`.

Inputs:

- `filename`: file name such as `topics.json`, `quizzes.json`, `exams.json`, or `summaries.json`.

Outputs:

- Absolute filesystem path string.

Why it exists:

- It prevents path issues when running the app from different working directories.

`load_json(filename, fallback)`

Purpose:

- Shared JSON reader for the data layer.

Inputs:

- `filename`: name of the JSON file inside `data/`.
- `fallback`: value returned if the file is missing or invalid.

Outputs:

- Parsed JSON object, usually a dictionary.
- Returns `fallback` on `FileNotFoundError` or `JSONDecodeError`.

Why it exists:

- Avoids repeating JSON loading code in each route.
- Makes future JSON resources easier to add.

`load_topics()`

Purpose:

- Loads topic data from `data/topics.json`.

Inputs:

- No arguments.

Outputs:

- Dictionary shaped like `{ "topics": [...] }`.
- Returns `{ "topics": [] }` if the file is missing or invalid.

Used by:

- `/`
- `/topics`

`load_quizzes()`

Purpose:

- Loads and validates quiz data from `data/quizzes.json`.

Inputs:

- No arguments.

Outputs:

- Dictionary shaped like `{ "quizzes": [...] }`.
- Only valid quiz objects are returned.

Validation:

- Each quiz must include `id`, `title`, `question_count`, `correct_answers`, and `pdfs`.

Special behavior:

- If `data/quizzes.json` does not exist, it creates an empty quiz file.

Used by:

- `/`
- `/homework`
- `/quiz/<quiz_id>`
- `/submit-quiz/<quiz_id>`

`load_exams()`

Purpose:

- Loads exam resource data from `data/exams.json`.

Inputs:

- No arguments.

Outputs:

- Dictionary shaped like `{ "exams": [...] }`.

Used by:

- `/`
- `/exams`

`load_summaries()`

Purpose:

- Loads summary PDF metadata from `data/summaries.json`.

Inputs:

- No arguments.

Outputs:

- Dictionary shaped like `{ "summaries": [...] }`.

Used by:

- `/`
- `/miscellaneous`

asset_url(path)

Purpose:

- Converts stored static paths into Flask-safe public URLs.

Inputs:

- path: string path such as `static/pdfs/topics_y1/T0-Introduccion.pdf`.

Outputs:

- URL string usable in an HTML href.

Behavior:

- Empty input returns an empty string.
- Absolute URLs starting with `http://`, `https://`, or `/` are returned unchanged.
- Paths starting with `static/` are converted with `url_for('static', filename=...)`.
- Other relative paths are also passed through `url_for('static', filename=...)`.

Used by:

- PDF links in templates.
- Home dashboard quick links.
- Reusable PDF macros.

inject_layout_context()

Purpose:

- Makes shared helpers and navigation data available to all templates.

Inputs:

- No direct input.

Outputs:

- Dictionary containing `asset_url` and `nav_items`.

Why it exists:

- Templates can call `asset_url(...)` without every route passing it manually.
- Navigation can be rendered from one central source.

5. Route Functions

home()

Route:

- /

Purpose:

- Renders the redesigned study dashboard.

Inputs:

- HTTP GET request.

Outputs:

- `home.html`

Template variables:

- `stats`: counts topics, quizzes, exams, and summaries.
- `recent_topics`: latest three topic entries.
- `featured_quizzes`: first three quiz entries.

login()

Route:

- /login

Methods:

- GET and POST.

Purpose:

- Shows the login form and authenticates users.

Inputs:

- GET: no form data.
- POST: username and password.

Outputs:

- GET returns `login.html`.
- Valid POST redirects to /.
- Invalid POST returns `login.html` with an error message.

Session effects:

- Sets `session['username']`.
- Sets `session['role']`.

logout()

Route:

- /logout

Purpose:

- Clears the session.

Inputs:

- HTTP GET request.

Outputs:

- Redirects to /.

topics()

Route:

- /topics

Purpose:

- Shows notes grouped by school year.

Inputs:

- Logged-in HTTP GET request.

Outputs:

- user/topics.html

Template variables:

- y1_pdfs: first-year topics.
- y2_pdfs: second-year topics.

homework()

Route:

- /homework

Purpose:

- Shows available quiz exercises.

Inputs:

- Logged-in HTTP GET request.

Outputs:

- `user/homework.html`

Template variables:

- `quizzes`: validated quiz list.

`take_quiz(quiz_id)`

Route:

- `/quiz/<quiz_id>`

Purpose:

- Shows answer choices for a selected quiz.

Inputs:

- `quiz_id`: route parameter from the URL.

Outputs:

- `user/quiz.html` if the quiz exists.
- `errors/404.html` if the quiz or question PDF is missing.
- `errors/500.html` if required quiz fields are missing.

Template variables:

- `quiz`: selected quiz object.

`submit_quiz(quiz_id)`

Route:

- `/submit-quiz/<quiz_id>`

Methods:

- POST.

Purpose:

- Processes submitted quiz answers.

Inputs:

- `quiz_id`: route parameter.
- Form fields named `answer_1`, `answer_2`, and so on.

Outputs:

- Redirects to `/quiz-results`.

Session effects:

- Stores `session['quiz_results']`.

Computed values:

- User answers.
- Correct answer count.
- Score percentage.
- Solution PDF path.

`quiz_results()`

Route:

- `/quiz-results`

Purpose:

- Displays the last submitted quiz result.

Inputs:

- Logged-in HTTP GET request.

Outputs:

- `user/quiz_results.html` if results exist.
- Redirects to `/homework` if no results are in session.

`miscellaneous()`

Route:

- `/miscellaneous`

Purpose:

- Shows PAU information, criteria, curriculum, educational approach, and summaries.

Inputs:

- Logged-in HTTP GET request.

Outputs:

- `user/miscellaneous.html`

Template variables:

- `y1_summaries`: first-year summaries.

- `y2_summaries`: second-year summaries.

`exams()`

Route:

- `/exams`

Purpose:

- Shows exam and PAU practice resources grouped by year.

Inputs:

- HTTP GET request.

Outputs:

- `user/exams.html`

Template variables:

- `y1_exams`: first-year exams.
- `y2_exams`: second-year exams.

Important note:

- This route remains public, matching the existing behavior. If desired, it can later receive `@login_required`.

6. New Data Files

`data/exams.json`

Purpose:

- Stores exam and PAU practice resource metadata that used to be hardcoded in `templates/user/exams.html`.

Top-level shape:

- `{ "exams": [...] }`

Each exam object contains:

- `id`: numeric identifier.
- `title`: visible title.
- `description`: card description.
- `url`: static PDF path.
- `year`: `y1` or `y2`.
- `type`: label such as `Simulacro`, `Correccion`, or `PAU`.

Inputs:

- Read by `load_exams()`.

Outputs:

- Rendered by `/exams` and counted by `/`.

Maintenance:

- Add new exam PDFs to `static/pdfs/examenes/`.
- Add matching metadata here.

`data/summaries.json`

Purpose:

- Stores summary PDF metadata that used to be hardcoded in the PAU/info template.

Top-level shape:

- `{ "summaries": [...] }`

Each summary object contains:

- `id`: numeric identifier.
- `title`: visible topic title.
- `description`: card description.
- `url`: static PDF path.
- `year`: y1 or y2.

Inputs:

- Read by `load_summaries()`.

Outputs:

- Rendered by `/miscellaneous` and counted by `/`.

7. New Template Component File

`templates/components/_macros.html`

Purpose:

- Provides reusable Jinja macros so pages can share consistent headers and PDF cards.

Macro: `page_header(eyebrow, title, copy, icon='fa-atom')`

Purpose:

- Renders a consistent page hero/header.

Inputs:

- `eyebrow`: small label above the title.
- `title`: main heading.
- `copy`: explanatory paragraph.
- `icon`: Font Awesome icon class. Defaults to `fa-atom`.

Outputs:

- HTML `<header>` block with consistent visual structure.

Used by:

- Topics page.
- Homework page.
- Exams page.
- PAU information page.

Macro: `pdf_card(item, tone='primary', icon='fa-file-pdf', badge='PDF')`

Purpose:

- Renders a reusable PDF resource card with view and download buttons.

Inputs:

- `item`: dictionary/object with at least `title`, `description`, and `url`.
- `tone`: visual style, usually `primary` or `beige`.
- `icon`: Font Awesome icon.
- `badge`: fallback badge label.

Outputs:

- HTML card with title, description, icon, View button, and Download button.

Dependencies:

- Calls `asset_url(item.url)`.

Used by:

- Topics page.
- Exams page.
- PAU summaries section.

8. New CSS Files

static/css/style.css

Purpose:

- Main CSS entry file loaded by `base.html`.

Behavior:

- Imports all split CSS files.

Outputs:

- Browser receives the full design system through one linked stylesheet.

static/css/tokens.css

Purpose:

- Defines design variables.

Includes:

- Primary blue: `#84b6f4`.
- Beige accent.
- Text colors.
- Background colors.
- Border radius values.
- Shadows.

Why it exists:

- Keeps colors and design constants in one place.

static/css/layout.css

Purpose:

- Controls global layout, page background, navbar, brand, main area, and footer.

Important classes:

- `.site-shell`
- `.site-main`
- `.site-nav`
- `.brand-mark`
- `.site-footer`
- `.footer-card`

Inputs:

- HTML class names from `base.html`.

Outputs:

- Global structure and navigation/footer styling.

`static/css/components.css`

Purpose:

- Defines reusable visual components.

Important classes:

- `.section-shell`
- `.page-hero`
- `.panel-card`
- `.resource-card`
- `.metric-card`
- `.resource-icon`
- `.badge-soft`
- `.action-row`
- `.error-icon`

Also customizes:

- Bootstrap primary buttons.
- Beige buttons.
- Form controls.
- Accordion active states.

`static/css/pages.css`

Purpose:

- Defines page-specific layouts.

Important classes:

- `.home-dashboard`
- `.dashboard-grid`
- `.study-path`
- `.study-path-item`
- `.study-path-number`
- `.home-link-card`
- `.course-heading`
- `.quiz-question`

- `.question-number`
- `.login-card`

`static/css/responsive.css`

Purpose:

- Handles tablet and mobile behavior.

Includes:

- Collapsed navigation spacing.
- One-column dashboard layout.
- Mobile footer layout.
- Full-width action buttons on small screens.
- Two-column quiz answer buttons on small screens.

9. Modified Templates

`templates/base.html`

Purpose:

- Provides the global HTML skeleton.

Main changes:

- New brand area.
- Direct navigation instead of one compact dropdown.
- Session-aware login/logout control.
- Shared footer.
- Uses split CSS through `static/css/style.css`.

Inputs:

- `nav_items` from `inject_layout_context()`.
- `session.username`.

Outputs:

- Shared layout for all pages.

`templates/home.html`

Purpose:

- Redesigned into a study dashboard.

Main changes:

- Hero section.
- Metrics for topics, quizzes, exams, and summaries.
- Recommended study path.
- Quick access cards.
- Recent topics.
- Featured quizzes.

Inputs:

- stats
- recent_topics
- featured_quizzes
- nav_items

Outputs:

- Dashboard page.

templates/user/topics.html

Purpose:

- Shows notes grouped by year.

Main changes:

- Uses page_header.
- Uses pdf_card.
- Removes inline CSS.
- Keeps Creative Commons notice.

Inputs:

- y1_pdfs
- y2_pdfs

templates/user/homework.html

Purpose:

- Shows quiz cards.

Main changes:

- Cleaner card layout.
- Consistent panel styling.
- Uses `asset_url()` for question PDFs.

Inputs:

- quizzes

`templates/user/quiz.html`

Purpose:

- Shows a selected quiz.

Main changes:

- New panel styling.
- Progress bar retained.
- JavaScript retained for enabling submit only after all questions are answered.

Inputs:

- quiz

Client-side behavior:

- Tracks checked radio buttons.
- Updates progress percentage.
- Enables the submit button when all questions are answered.

`templates/user/quiz_results.html`

Purpose:

- Shows quiz score and answer review.

Main changes:

- New result header.
- Score metric card.
- Cleaner answer review layout.
- Uses `asset_url()` for solution PDF.

Inputs:

- results

`templates/user/exams.html`

Purpose:

- Shows exam PDFs grouped by year.

Main changes:

- No longer hardcodes each exam.
- Receives y1_exams and y2_exams from Flask.
- Uses pdf_card.
- Fixes broken PDF reference by using data/exams.json.

Inputs:

- y1_exams
- y2_exams

templates/user/miscellaneous.html

Purpose:

- PAU reference page.

Main changes:

- Reorganized into structured sections.
- Keeps exam structure, evaluation criteria, important considerations, curriculum, educational approach, and summaries.
- Summary PDFs now come from JSON.

Inputs:

- y1_summaries
- y2_summaries

templates/login.html

Purpose:

- Login form.

Main changes:

- Matches the new visual system.
- Keeps the same POST behavior and field names.

Inputs:

- Optional error.

Form outputs:

- POSTs username and password to /login.

Error templates

Files:

- templates/errors/403.html
- templates/errors/404.html
- templates/errors/500.html

Purpose:

- Consistent error pages.

Inputs:

- 404.html can receive message.
- 500.html can receive error.

Outputs:

- Styled error screen with link back to home.

10. Documentation, Local Tooling, Tests, and Ignore Files

.gitignore

Purpose:

- Prevents local generated files from being committed.
- Keeps local virtual environments out of Git.
- Keeps generated pytest and Python bytecode artifacts out of the repository.

Ignored examples:

- Python cache files.
- .venv/
- venv/
- .env
- .pytest_cache/
- pytest-cache-files-*/
- logs.
- Codex temporary files.
- .DS_Store
- Thumbs.db

README.md

Purpose:

- Documents how the project works and how to run it.

Includes:

- Flask explanation.
- Project structure.
- Local setup.
- Test users.
- Summary of redesign changes.
- Validation notes.
- Render deployment notes.
- Future recommendations.

AGENTS.md

Purpose:

- Defines repository-specific rules for AI coding agents.
- Protects the core architecture: Flask, Jinja templates, JSON content, static PDFs, and Render compatibility.
- Requires small, understandable changes and progress-tracker updates after completed work.

context/

Purpose:

- Provides durable project context for future AI-assisted work.
- Keeps product scope, architecture, code standards, workflow rules, UI context, progress tracking, and feature-spec guidance in one place.

Important files:

- context/01-project-overview.md
- context/02-architecture.md
- context/03-code-standards.md
- context/04-ai-workflow-rules.md
- context/05-ui-context.md
- context/06-progress-tracker.md
- context/feature-specs/README.md

Local Windows scripts

Files:

- start-web.bat
- run-local.ps1
- setup-local.ps1

Purpose:

- Help a non-expert Windows maintainer set up and run the Flask app locally.
- Prefer the ignored .venv/ environment.

- Keep local startup simple without changing production deployment assumptions.

Pytest smoke tests

Files:

- `pytest.ini`
- `tests/test_app_smoke.py`

Purpose:

- Verifies the app imports.
- Checks key public routes.
- Checks protected routes redirect or deny access before login.
- Confirms `/exams` remains public.
- Confirms JSON files load.
- Checks main page rendering does not crash.

Important test setting:

- `pytest.ini` disables pytest's cache provider with `-p no:cacheprovider` to avoid local cache write issues on this Windows/OneDrive path.

11. Validation Completed

The following checks have been used across the consolidation work and remain the recommended validation set:

- Git working tree sanity check.
- Flask import through `.venv`.
- Flask route list inspection.
- Flask test client checks for public routes.
- Login test with Guest.
- Flask test client checks for protected routes after login.
- Jinja template syntax parse.
- JSON parse for every file in `data/`.
- CSS import existence checks.
- Static JSON link existence checks.
- Pytest smoke tests through `.\.venv\Scripts\python.exe -m pytest`.

Observed route results:

- `/`: 200
- `/login`: 200
- `/exams`: 200
- `/topics` after login: 200
- `/homework` after login: 200

- `/miscellaneous` after login: 200

Current smoke test expectation:

- `pytest.ini` should discover `tests/test_app_smoke.py`.
- The smoke suite should pass without creating committed pytest cache artifacts.

12. Important Decisions

No database was introduced

Reason:

- The user specifically requested keeping JSON data for now.
- JSON is sufficient for current topics, quizzes, exams, and summaries.

No React or frontend framework was introduced

Reason:

- The project intentionally remains Flask plus templates.
- The redesign uses Bootstrap, Jinja, and custom CSS.

Colors were preserved

Reason:

- The user requested keeping the existing colors.
- The primary blue remains `#84b6f4`.
- Beige remains the secondary course accent.

exams remains public

Reason:

- This matches the previous behavior.
- Other student resource pages remain protected.

Possible future change:

- Add `@login_required` to `/exams` if all study content should require login.

13. Future Improvement Ideas

- Expand pytest coverage for quiz submission, error pages, and broken PDF links.
- Move users out of `app.py` and into environment variables or a safer auth system.
- Add stricter data validation for every JSON file.

- Add a progress tracking model when a database is introduced.
- Convert hardcoded PAU curriculum blocks into JSON if they grow.
- Add CI checks for Jinja syntax, JSON validity, and broken static links.
- Add a Render Profile if Render configuration is not already handled in the dashboard.